

Guía FP Ejercicio DO WHILE (Repetir Hasta Que)

Triángulo rectángulo hueco de 10 líneas

Propósito de la guía. Acompañarte paso a paso para desarrollar el ejercicio con estructura repetitiva DO WHILE, primero en PSeInt y luego en Code::Blocks, cuidando la lógica, la sintaxis y la prueba de escritorio. La idea es que no solo funcione, sino que entiendas por qué funciona.

Resultado esperado

Imprimir un triángulo rectángulo hueco de 10 líneas usando asteriscos (*), dejando vacío el interior.

Contexto del ejercicio

En el documento base se presenta un programa en C que solicita el número de líneas y dibuja un triángulo hueco. La condición clave es imprimir asterisco cuando la posición esté en el borde izquierdo, en la diagonal o en la última fila; caso contrario se imprime espacio.

Regla de impresión: si $j = 1$, $j = i$ o $i = n$, se imprime "*"; en cualquier otro caso se imprime un espacio.

1. Seudocódigo para PSeInt

Antes de programar en C, conviene escribir la solución en PSeInt. Aquí se observa la lógica con palabras cercanas al lenguaje natural.

Proceso TrianguloHueco

Definir n, i, j Como Entero

Repetir

 Escribir "Lineas del triangulo: "

 Leer n

Hasta Que $n > 0$

Para i <- 1 Hasta n Con Paso 1 Hacer

 Para j <- 1 Hasta i Con Paso 1 Hacer

 Si $j = 1$ O $j = i$ O $i = n$ Entonces

 Escribir Sin Saltar "*" "

 SiNo

 Escribir Sin Saltar " "

 FinSi

FinPara

Escribir ""

FinPara

FinProceso

Tip amigable: imagina que i representa la fila y j representa la columna. El programa revisa casilla por casilla para decidir si coloca un asterisco o un espacio.

2. Prueba de escritorio en PSeInt

La prueba de escritorio permite comprobar manualmente qué hace el algoritmo. Para que sea fácil de revisar, se muestra el comportamiento cuando $n = 10$.

Fila i	Rango de j	Condición que dibuja borde	Salida de la fila	Explicación breve
1	1	$j=1$ y $j=i$	*	Primera fila: solo un asterisco.
2	1 a 2	$j=1$ o $j=i$	**	Tiene borde izquierdo y diagonal.
3	1 a 3	$j=1$ o $j=i$	* *	El centro queda vacío.
4	1 a 4	$j=1$ o $j=i$	* * *	Aumenta el espacio interno.
5	1 a 5	$j=1$ o $j=i$	* * *	Se mantienen los bordes.
6	1 a 6	$j=1$ o $j=i$	* * *	Continúa el patrón hueco.
7	1 a 7	$j=1$ o $j=i$	* * *	Más columnas internas vacías.
8	1 a 8	$j=1$ o $j=i$	* * *	El borde diagonal se desplaza.
9	1 a 9	$j=1$ o $j=i$	* * *	Última fila antes de la base.
10	1 a 10	$i=n$	*****	Base completa del triángulo.

Salida esperada al ingresar 10:

Lineas del triangulo: 10

```

*
* *
*  *
*   *
*    *
*     *
*      *
*       *
*        *
* * * * *

```

3. Guía para pasar de PSeInt a Code::Blocks con sintaxis validada

Ahora sí: pasamos la lógica a lenguaje C. La clave es traducir cada instrucción de PSeInt a su equivalente en C, sin perder el orden de los ciclos.

PSeInt	C / Code::Blocks	Para recordar
Proceso ... FinProceso	int main() { ... return 0; }	Todo programa en C inicia en main.
Definir i, j, n Como Entero	int i, j, n;	Las variables enteras se declaran con int.
Escribir	printf();	printf muestra texto o resultados.
Leer n	scanf("%d", &n);	El & indica la dirección donde se guarda el dato.
Repetir... Hasta que n > 0	Do {...} while (n <= 0)	Primero ejecuta y luego evalúa la condición. Sirve para validar que el número de líneas sea positivo.
Para i <- 1 Hasta n	for (i = 1; i <= n; i++)	El FOR tiene inicio, condición e incremento.
Si ... Entonces	if (...)	La condición usa operadores lógicos.
O		En C, OR se escribe con doble barra vertical.
= para comparar	==	En C, == compara; = asigna.
Escribir Sin Saltar	printf("...");	No se coloca \n si se quiere seguir en la misma línea.

Código validado en C para Code::Blocks

// Genera un triangulo hueco de acuerdo al numero de lineas con do while y for

#include <stdio.h>

```
int main()
{
    int i, j, n;

    // Validacion del numero de lineas
    do
    {
        printf("Lineas del triangulo: ");
        scanf("%d", &n);

    } while(n <= 0);

    // Control de filas
    for(i = 1; i <= n; i++)
    {
        // Control de columnas
        for(j = 1; j <= i; j++)
        {
            // Bordes y base
            if(j == 1 || j == i || i == n)
            {
                printf("* ");
            }
            else
            {
                printf(" ");
            }
        }

        printf("\n");
    }
}
```

```

    }

    return 0;
}

```

Pasos sugeridos en Code::Blocks

1. Crear un nuevo archivo con extensión .c o abrir el archivo del ejercicio.
2. Copiar el código validado en C.
3. Compilar con Build o F9
4. Ejecutar con Run o con Build and Run.F9
5. Cuando el programa solicite el número de líneas, escribir 5, 6,... 10 y presionar Enter.
6. Comparar la salida con la prueba de escritorio.

Errores comunes que debes evitar:

- Usar `=` en lugar de `==` dentro del if.
- Olvidar el símbolo `&` en `scanf("%d", &n)`.
- Cambiar `||` por la palabra `O`; en C debe escribirse `||`.
- Colocar `printf("\n")` dentro del segundo FOR; eso rompería la forma del triángulo.
- Imprimir un solo espacio en el else; para mantener la forma se recomienda imprimir dos espacios.

4. Rúbrica de evaluación sobre 20 puntos

Esta rúbrica permite valorar el desarrollo completo del ejercicio, desde la comprensión del problema hasta la ejecución en Code::Blocks.

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Identifica claramente que el triángulo debe ser hueco, con borde izquierdo, diagonal y base.	Reconoce la forma del triángulo, aunque explica parcialmente la condición de impresión.	Confunde triángulo lleno con triángulo hueco o no distingue los bordes.	4 pts
Seudocódigo en PSeInt	Usa correctamente variables, Repetir, Para, Si/Sino y Escribir Sin Saltar.	Presenta la estructura general, con pequeños errores de sintaxis o ubicación.	El pseudocódigo está incompleto o no representa la lógica del ejercicio.	4 pts
Prueba de escritorio	Comprueba el algoritmo fila por fila y explica la salida para $n = 10$.	Realiza una prueba parcial con algunas filas o poca explicación.	No evidencia seguimiento de variables i y j o la salida no coincide.	4 pts
Traducción a C en Code::Blocks	El código compila, ejecuta y mantiene la lógica validada del triángulo hueco.	El código tiene errores menores corregibles, pero la lógica principal es adecuada.	El código no compila o no aplica correctamente FOR, if y operadores lógicos.	5 pts
Presentación y orden	Entrega el trabajo limpio, ordenado y fácil de leer, con comentarios útiles.	El trabajo es comprensible, aunque puede mejorar en orden o comentarios.	El trabajo es difícil de seguir o no evidencia organización.	3 pts

Total: 20 puntos. Para una buena calificación, lo más importante es que exista coherencia entre pseudocódigo, prueba de escritorio, código en C y salida final.

Lista rápida de verificación antes de entregar

- El programa solicita el número de líneas.
- El valor ingresado se valida usando Repetir para que sea mayor que 0.

- El primer FOR controla las filas.
- El segundo FOR controla las columnas de cada fila.
- El if imprime asterisco en borde izquierdo, diagonal y base.
- El else imprime espacios para dejar el interior hueco.
- Al ingresar 10, se observan 10 líneas en pantalla.